TALENTRA DATABASE DESIGN HACKATHON

Milestone 10 — Keys and Constraints (B33–B35)

Status: Proposed implementation and validation draft. Must be reconciled with approved Milestone 7 entity names and Milestone 8 relationships, then tested and accepted by the team.

1. Scope from the docket

B33: Administration keys and constraints, including role-based access enforcement and append-only audit trail. B34: Organisation and account keys and constraints, including effective-dated recruiter mapping. B35: Gig marketplace keys and constraints, including the rule that gig windows cannot overlap for the same gig worker.

2. Shared baseline

Rule	Implementation baseline
Primary keys	Every entity has a surrogate PK named id, using one consistent type. Example uses BIGINT identity if PostgreSQL is the agreed engine.
Foreign keys	Enforce in the database; document ON UPDATE and ON DELETE for every FK. RESTRICT is a conservative default for retained business/audit history; confirm per relationship.
Unique constraints	Use UNIQUE for natural identifiers and mapping pairs when duplicates are prohibited. Constraint names follow uq_table_column.
Check constraints	Use CHECK for date ranges, nonnegative amounts, valid score ranges, and other row-level business rules. Names follow a consistent chk_table_rule pattern.
Indexes	Add indexes for important FK joins and expected query patterns; names follow idx_table_column. Avoid duplicating an existing unique index.
Audit and deletion	Business tables have created_at, created_by, updated_at, updated_by. State soft/hard-delete policy and deleted_at when soft deletion is used. Audit history is append-only.
Caveat	Rules that span rows/tables—such as non-overlapping time ranges—usually need exclusion constraints or transactional enforcement, not a simple CHECK.

3. B33 — Administration

Entity / concern	Proposed keys and constraints
app_user	PK(id). UNIQUE(lowercase email) only if email identifies a single account; unique phone only if product rules require it. CHECK or lookup FK for account_status_code. ON DELETE RESTRICT where sessions/audit history must remain.
credential	PK(id); FK user_id → app_user(id). Unique credential identity by credential_type_code plus provider_subject or normalized login identifier, as appropriate. CHECK exactly the appropriate identity field is populated for each credential type. Never store plaintext secrets.
role / permission	PK(id); UNIQUE(code); CHECK sort_order >= 0. Keep active/status as controlled lookup/boolean according to the approved dictionary.
user_role	PK(id); FK user_id and role_id. UNIQUE(user_id, role_id) if duplicate grants are not allowed. Consider valid_from/valid_to if role assignment history is needed.
role_permission	PK(id); FK role_id and permission_id. UNIQUE(role_id, permission_id) to prevent duplicate permission grants.
user_session	PK(id); FK user_id. CHECK ended_at IS NULL OR ended_at >= started_at. Define active-session predicate and device-limit enforcement at login using transaction/locking; record denials safely.
consent_record	PK(id); FK user_id. CHECK withdrawn_at IS NULL OR withdrawn_at >= recorded_at. Preserve consent history according to retention policy.
audit_event	PK(id); nullable actor FK only if system events are allowed. Append-only: restrict UPDATE/DELETE through database privileges and operational controls; index occurred_at, actor_user_id, entity reference/correlation ID as query needs.
block_entry	PK(id); FKs to blocker and blocked users. CHECK blocker_user_id <> blocked_user_id. UNIQUE(blocker_user_id, blocked_user_id) if only one current block per pair is allowed.

4. B34 — Organisation and account

Entity / concern	Proposed keys and constraints
organisation	PK(id); FK industry_id to Master Data if that lookup exists. CHECK legal_name is not blank; status code references approved lookup. Natural registration IDs should be unique only within agreed country/type scope.
organisation_location	PK(id); FK organisation_id; FKs to city/state/country lookups where approved. Decide whether address fields are required by location type; do not invent universal uniqueness for addresses.
organisation_identifier	PK(id); FK organisation_id and country_id. UNIQUE(country_id, identifier_type_code, normalized identifier_value) only if the identifier is globally unique in that jurisdiction/type; sensitive values need masking/protection.
organisation_verification	PK(id); FK organisation_id; FK verified_by to user if actor identity is in scope. CHECK verified_at IS NULL OR status is a verified terminal state, if status semantics support it.
recruiter_organisation_mapping	PK(id); FKs organisation_id and recruiter_user_id. CHECK valid_to IS NULL OR valid_to > valid_from. Prevent overlapping active mappings for same recruiter/organisation if disallowed; PostgreSQL range + EXCLUDE constraint is one option.
account	PK(id); UNIQUE(account_code). FK organisation_id if relationship is approved. Define ON DELETE and whether an account can exist without an organisation.
account_admin	PK(id); FKs account_id and user_id. UNIQUE(account_id, user_id) if only one current mapping is stored; for history, use effective dates and enforce non-overlap if required.
organisation_hierarchy	PK(id); FKs parent_organisation_id and child_organisation_id. CHECK parent <> child. Cycle prevention requires more than a simple CHECK; use transactional logic/trigger or closure-table approach if tree semantics are required.

5. B35 — Gig marketplace

Entity / concern	Proposed keys and constraints
gig_worker_profile	PK(id); FK user_id. UNIQUE(user_id) only if each user has at most one gig worker profile.
gig_service	PK(id); FK gig_worker_profile_id. Add UNIQUE(worker_id, service_code) only if duplicate services are not allowed.
gig_availability	PK(id); FK gig_worker_profile_id. CHECK ends_at > starts_at. If availability windows must not overlap, define whether adjacent boundaries are allowed and implement exclusion/transactional enforcement.
gig_request	PK(id); FKs requester_user_id and/or organisation_id as approved. CHECK ends_at > starts_at. CHECK price_amount >= 0 when supplied; require currency_code when price_amount is non-NULL. Decide whether requester must be exactly one of user/organisation.
gig_application	PK(id); FKs gig_request_id and gig_worker_profile_id. UNIQUE(gig_request_id, gig_worker_profile_id) if only one application per worker/request is allowed.
gig_assignment	PK(id); FK gig_application_id. UNIQUE(gig_application_id) if one assignment per accepted application. CHECK ends_at > starts_at; amount nonnegative; currency required with amount.
gig_handoff	PK(id); FK gig_assignment_id. UNIQUE(external_system_code, external_reference) where external reference is idempotency identity. Do not store secrets in metadata.
gig_earnings_snapshot	PK(id); FK gig_assignment_id. CHECK amount >= 0; currency required; source_reference uniqueness only if external source guarantees uniqueness.
gig_worker_list_entry	PK(id); FKs owner_user_id and gig_worker_profile_id. UNIQUE(owner_user_id, gig_worker_profile_id) if duplicate saved entries are disallowed.
gig_rating	PK(id); FKs gig_assignment_id, reviewer_user_id and rated_user_id. CHECK score between the agreed minimum and maximum. Prevent self-rating if prohibited; enforce one rating per reviewer/assignment if that is the approved rule.

6. Gig-window non-overlap requirement

The docket explicitly requires that gig windows cannot overlap for the same gig worker. Decide which records count as a gig window (availability, assignment, or both), whether touching endpoints are allowed, and how cancelled/completed records are treated. A simple row-level CHECK cannot compare a new row with other rows. If PostgreSQL is selected, an EXCLUDE constraint using a time range and worker identifier can prevent overlapping assignments; otherwise use a transaction-safe locking strategy. Test concurrent inserts, not just sequential ones.

7. Milestone 10 validation checklist

• All entity/table names and columns match the approved Milestones 7–9 inventories.

• Every entity has a surrogate PK; all FKs are enforced and have explicit ON UPDATE/ON DELETE actions.

• Natural/business uniqueness rules are agreed, named, and tested for duplicates.

• CHECK constraints are tested with valid and invalid values; multi-row rules use suitable database mechanisms.

• Role-based access controls are enforced consistently; audit events cannot be silently updated/deleted by ordinary application roles.

• Effective-dated recruiter mapping has a documented overlap policy and a tested enforcement method.

• Gig time-window overlap is prevented under concurrent requests, with clear boundary and cancellation semantics.

• DDL applies to a clean database; test evidence, open risks, approver names, and date are recorded.

• B33, B34, and B35 are reviewed by their owners; only then can the team sign off the Phase 2 module constraints.

Source vs proposal: The three B33–B35 topics and the gig non-overlap requirement are stated in the docket. Specific constraint choices in this document are proposed implementation patterns and require approval against the final schema and business rules.